

Guided Exploration M03: Loops, Methods and Testing

Points: 25 (See Rubric in Canvas)

Due Date: Due date listed in Canvas but some sections will be due as class participation before. Create Calendar reminders

- Final submission will be accepted up to 24 hours after the due date with a 10% penalty. Meaning if you turn it in at 12:01 am of the next day you will be deducted 10% of the total points from your score.
- If the assignment is more than 24 hours late, it will be a 0.

Submission: Upload files separately and do not upload a zip file.

- **Two Document Files as PDF or Word document:**

- This document with your answers in the highlighted areas
- Technical Document.

- **One Java Files (. java files NOT CLASS FILES):** Code from

- [2.2 Practice Software Development Life Cycle](#)

Objectives: See objectives in canvas for module 3. Apply concepts from module 1 through 3. Do not use arrays or create your own classes.

Effort: You are encouraged to collaborate to discuss concepts and explore writing code together. Remember

- Review lectures and links in the lectures to information first.
- Follow [CS Academic Integrity and AI Policy - Harding](#)
- If you find you don't understand the information in the lecture reach out to **Deb, Elaine or Heriberto**

[Part 1: Explore and Explain](#)

[Part 2: Apply](#)

[2.1 Calculate Sum](#)

[2.2 Software Development Life Cycle](#)

[Requirements: User Stories and Acceptance Criteria](#)

[Part 1: Break Into Tasks](#)

[Part 2: Unit Test Cases, Design Algorithm, Implement, Debug](#)

[3 Learnings and Reflection](#)

[Advanced Exploration](#)

Part 1: Explore and Explain

Open your tech document.

- Add a new Title for GE03 and subtitle for the different lectures

- Go back to the lectures and the code I had you explore in class.
- Practice writing code and include comments.
- Summarize the concepts and copy and paste your example code into your tech document.

Objectives:

- **Understand and Explain Loops and Nested Control Structures**
 - different types of loops: **for**, **while**, **do-while**
 - When to use the different loops
 - different ways to exit a while loop
 - ways you should not use to exit a loop
 - Sentinel values and boolean flags
 - Scope of variables in loops
 - Nested loops and conditions
- **Understand and Explain Methods**
 - Method signature: Method Name, Parameters and Return Value
 - Write and call your own methods you define.
 - With and without parameters
 - With and without return value
 - Each method should only have one return value at the end of the method
 - Scope of variables in methods
 - Stack Frame Memory when calling methods and Pass by Value

Part 2: Apply

Implement Control Structures and Methods to Write Modular Solutions in Java

- Implement loops and conditions and nested control structures to control the flow of a program
- Implement **methods** to modularize repetitive code and follow single responsibility principle and do not repeat your self

2.1 Calculate Sum

Plan Test Cases, Design Algorithm, Implement and Unit Test Code to sum the integer values from 1 to a max value.

1. Write test cases to find the sum of the numbers 1 to a max. For example if max is 4 then the sum is 10 because $1 + 2 + 3 + 4 = 10$
I included one for you.

Fill in coding notes to help you find a pattern

Precondition	Postcondition	Coding Notes
--------------	---------------	--------------

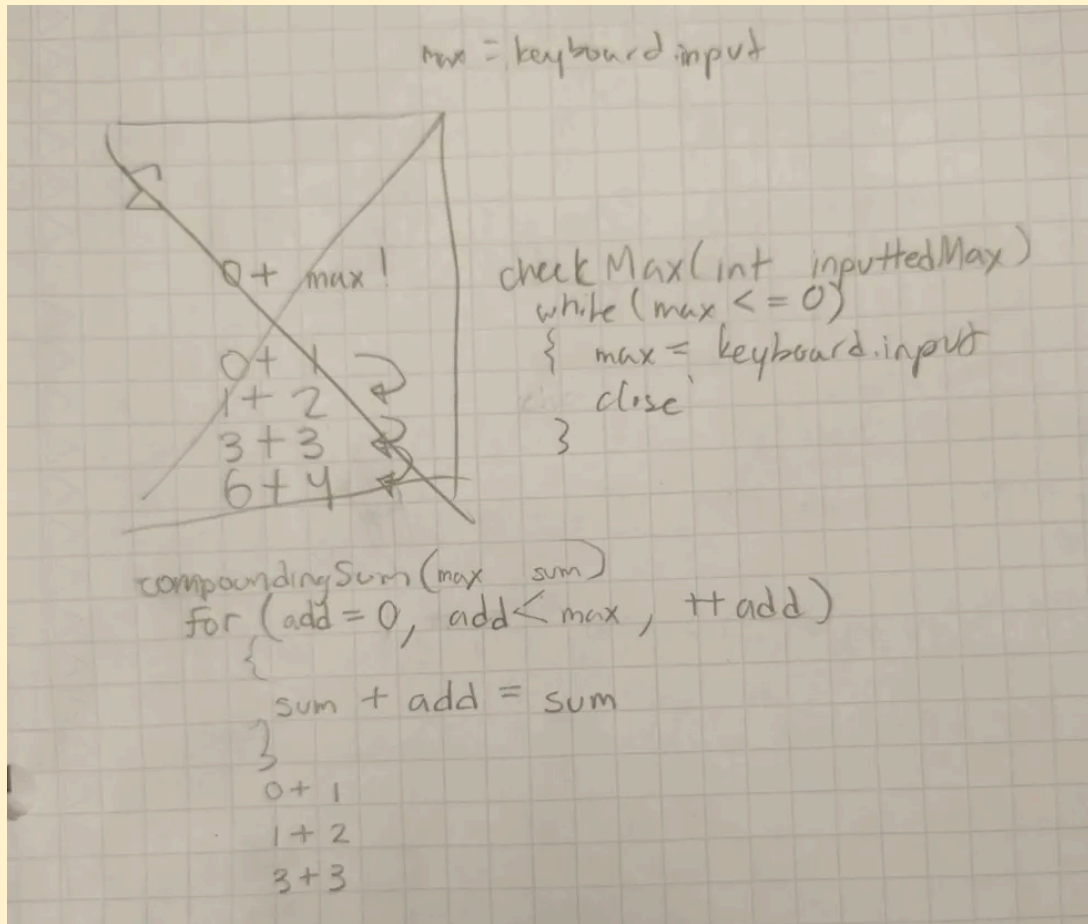
max = 4 sum = 0	sum = 10	sum + 1 sum + 2 sum + 3 sum + 4
max = 6 sum = 0	sum = 21	sum + 1 = 1 sum + 2 = 1 + 2 = 3 sum + 3 = 3 + 3 = 6 sum + 4 = 6 + 4 = 10 sum + 5 = 10 + 5 = 15 sum + 6 = 15 + 6 = 21
max = 0 sum = 0	sum = 0	sum + 0

Method Name using verbs: calculateSum
Method Description: sum the integer values from 1 to a max value.
Parameters (What does the method need) : the int max value
Return (What does the method return);: an int for sum
Method Signature (return type, method name, parameters): int
 CalculateSum(int currentMax)

2. Design your algorithm to solve the problem.

What loop do you think you should use?

Put the image below of the handwritten pseudocode for the algorithm on paper, white board or tablet for pseudocode to get credit.



3. Method Information

What information must be stored in main?

At least two variables for max and sum should be stored in main so that the methods can pass them, and return their changed values.

What must be passed to the calculateSum method?

Both max and sum should be passed to calculateSum.

What is returned to main when the calculateSum method completes?

Calculate sum returns the variable integer completeSum.

4. Write the code in Eclipse and test it. Use the debugger to see what the code is doing. Remember to commit and push.

Take a screenshot showing what is in the debugger after stepping through two iterations of the loop.

Name	Value
no method return	
args	String[0] (id=20)
sum	0
keyboardInput	Scanner (id=22)
max	6

Name	Value
no method return	
inputtedMax	6

Name	Value
no method return	
completeSum	21
initializedMax	6
add	6

Name	Value
no method return	
completeSum	21
initializedMax	6

Name	Value
no method return	
args	String[0] (id=20)
sum	21
keyboardInput	Scanner (id=23)
max	6

5. Method Stack Frame and Pass by Value

What are the local variables in the main method?

Local variables sum and max are stored in the main method.

When the calculateSum method is called, what happens in memory?

In memory, main calls the calculateSum method, pushing it into the stack.

What are the local variables in the calculateSum method?

The local variables in calculateSum are the ints completeSum and initializedMax.

How is information passed and stored?

Information is passed by first allocating memory via parameters in the first method, those parameters allow you to store copies of the variables from other methods, but they must be declared within the parameters to allocate memory. After the method is completed, the memory is unallocated.

Why would changing the currentMax parameter value inside the method not change the original variable in main?

Because currentMax is just a copy of the original variable in main.

When the calculateSum method finishes, what happens to its local variables?

The calculateSum method's local variables are unallocated from memory.

When the main method finishes, what happens to its local variables?

The main method's local variables remain there.

Draw the stages of what happens when calling this method from main.

Use boxes to show stack frames for each method and include local variables for each stack frame for each of the following stages. Draw arrows to show what is passed and returned.

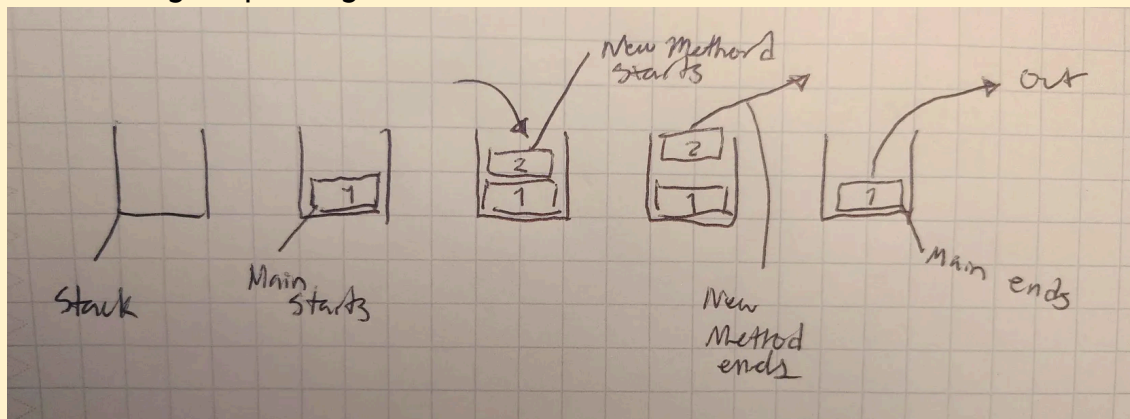
Stage 1. Main Method Stack Frame Before the Method Call

Stage 2. Stack Frame During the Method Call

Stage 3. Returning From the Method

Stage 4. Main method ends

Put the image explaining stack frames below.



7. Update your technical document with summaries and examples from above.

No need to turn in your java file.

2.2 Software Development Life Cycle

Follow [CS1050 Agile Software Development](#)

1. Analyze requirements and break into smaller tasks where each has a single responsibility and you do not repeat unnecessary to design your methods
2. Develop Unit tests for the tasks (methods) to help you design the algorithm
 - a. Create preconditions and postconditions and algorithm notes
 - b. Write algorithm in pseudocode on paper
3. Determine method signature - name, parameters and return values
4. Create the method and call from main, debug and unit test that it works.
5. Repeat for next task
6. Commit and push

Requirements: User Stories and Acceptance Criteria

User story 1: View Program Summary

As a trainer, I want to see a summary of what the program does and the BMI category ranges so that I understand how BMI will be interpreted.

Acceptance Criteria:

Display the following:

Team and Athlete Analysis

The trainer enters athlete data until they indicate they are done entering data.

The trainer will enter each athlete's weight and height

For each athlete entered, the BMI value and category will be displayed based on these BMI ranges.

Under 18.5: Underweight

18.5 to under 25: Normal

25 to under 30: Overweight

30 or greater: Obese

User Story 2: Validate Athlete height and weight

As a trainer, I need a fitness tracking system to enter each athlete's fitness data (weight, height, age) so the program can correctly calculate BMI and Max Heart Rate.

- Weight and height can contain decimal values
- height and weight must be > 0
- Displays error and repeats until valid

User Story 3: Calculate BMI

As a trainer, I want the program to calculate each athlete's BMI .

- BMI formula used: $703 \times \text{weight} / \text{height}^2$

User Story 4: Display Athlete BMI Value and Category

As a trainer, I want the program to display each athlete's BMI so that I can review individual results.

Acceptance Criteria

- Display the BMI numeric value to 1 decimal place and the correct category
- If the BMI indicates underweight (BMI <18.5) display message "Needs Review"

User Story 5: Allow Multiple Entries

As a trainer I want to be able to add more athlete data so that I can enter as many athletes as needed.

Acceptance Criteria

- Ask if another athlete should be entered (y/n)
- Check that valid data has been entered: 'y', 'Y', 'N' or 'n'
 - a. When not a y or n character repeat question

- b. 'y' or 'Y' repeat above steps for next athlete
- c. 'n' or 'N' Display the number of athletes that entered and ended the program. For example "4 athlete calculations were completed.

Part 1: Break Into Tasks

List tasks as verbs.

Task to Display summary

Fill in remaining tasks below

Task 1:

- Task to Display summary
- Display BMI formula

Task 2, setting up data:

- Input BMI
- Check Valid Inputs

Task 3:

- Calculate BMI
- Display proper BMI Range
- Implement "Needs Review" If BMI Underweight

Task 4

- Repeat Athletes

What tasks use the same algorithm and data type?

Strings/Ranges:

Display BMI Ranges

Implement "Needs Review" If BMI Underweight

Strings/BMI:

Display BMI Ranges

Display BMI formula

Int/Double:

CalculateBMI

Doubles:

Input BMI

Doubles/CheckLoop:

Check Valid Inputs

Char/Boolean while-if:

Repeat Athletes

How can you design your solution to not write duplicated code?

Integrate methods that can be used over and over again to avoid extreme repetition.

List constants and variables with data types that are needed

Double, int, boolean, char, String

Weight, height, BMI_US_FACTOR, bmi, continueYN, bmiCategory

Part 2: Unit Test Cases, Design Algorithm, Implement, Debug

Design Unit Tests and Method algorithms

- Unit tests
 - Write Pre/Post conditions with actual values that will be tested
- Method signature: return type, method name, parameters
 - **Method name: use verbs and correct naming conventions**
 - **Parameters: What does the method need**
 - **Return: What data type does the method return**
- Handwritten pseudocode for algorithms must be on paper, white board or tablet for pseudocode to get credit.
- Implement code and test - Do not worry if you are not able to get all your code working but you should have all of your test cases and design of algorithms completed.

You can use your work from previous guided explorations if you did it correctly when completing the following.

User story 1 Method to Display Summary

Method Name: printSummary

Description: Display the program summary.

Parameters: none

Return: none

Method Signature: public static void printSummary()

No need for test cases or algorithm

Create a new project called TrainerProject

Create a class

Implement the method and call the method from main.

Version Control:

Commit with a summary of what you completed

Push and put a screenshot below of your code added to the github remote server. Do this for every method you add.

```
declare and initialize some variables for methods
P main

EclipseWorkspace/CS1050M03
  TrainerProject.java

1 file changed +11 0 lines changed
EclipseWorkspace/CS1050M03/TrainerProject.java
+++ @@ -1,3 +1,5 @@
1 + import java.util.Scanner;
2 +
3
4 public class TrainerProject
5 {
6
7     @@ -6,6 +6,11 @@ public static void main(String[] args)
8     // prints program summary
9     printSummary();
10
11 +
12 + // creates a scanner
13 + Scanner keyboardInput = new Scanner(System.in);
14 +
15 + // declared and initialized variables to assign later
16 + double weight = 0;
17 + double height = 0;
18 + double bmi = 0;
19 + boolean continueProcess;
20 +
21 +
22 }
23
24 public static void printSummary()
```

```
checks inputs for bmi calculation
so that there are no impossible bmis
P main

EclipseWorkspace/CS1050M03
  TrainerProject.java

1 file changed +10 0 lines changed
EclipseWorkspace/CS1050M03/TrainerProject.java
+++ @@ -41,6 +41,16 @@ public static void printSummary()
41
42 public static double getPositiveDouble(Scanner input, double bmiInput)
43 {
44     // Initializing bmiInput
45     bmiInput = input.nextDouble();
46
47     // while the height is negative or 0, allows you to input again.
48     while (bmiInput <= 0)
49     {
50         System.out.println("Invalid input, cannot be negative or 0. Enter again: ");
51         bmiInput = input.nextDouble();
52     }
53     // passes value as the output of this method
54     return bmiInput;
55 }
56 }
```

```
Created calculateBmi
P main

EclipseWorkspace/CS1050M03
  TrainerProject.java

1 file changed +12 0 lines changed
EclipseWorkspace/CS1050M03/TrainerProject.java
+++ @@ -27,6 +27,9 @@ public static void main(String[] args)
27
28 System.out.println("Enter your height, in inches: ");
29
30 height = getPositiveDouble(keyboardInput, height);
31
32 +
33 + // calculate BMI/print
34 + bmi = calculateBMI(weight, height);
35 +
36 }
37
38 public static void printSummary()
39 {
40     @@ -31,4 +31,13 @@ public static double getPositiveDouble(Scanner input, double bmiInput)
41     // passes value as the output of this method
42     return bmiInput;
43 }
44
45 +
46 + public static double calculateBMI(double weightChosen, double heightChosen)
47 + {
48     // declared variable
49     double bmiCalculated;
50
51     // calculated bmi formula
52     bmiCalculated = weightChosen * 703 / Math.pow(heightChosen, 2);
53
54     return bmiCalculated;
55 }
```

```
prints calculatedBMI for user
P main

EclipseWorkspace/CS1050M03
  TrainerProject.java

1 file changed +5 0 lines changed
EclipseWorkspace/CS1050M03/TrainerProject.java
+++ @@ -61,6 +61,11 @@ public static double calculateBMI(double weightChosen, double heightChosen)
61
62 {
63     // declared variable
64     double bmiCalculated;
65
66     // calculated bmi formula
67     bmiCalculated = weightChosen * 703 / Math.pow(heightChosen, 2);
68
69     // prints bmi
70     System.out.println("Your BMI is " + bmiCalculated + ".");
71
72     return bmiCalculated;
73 }
```

```
determines BMICategory
created getBMICategory
P main

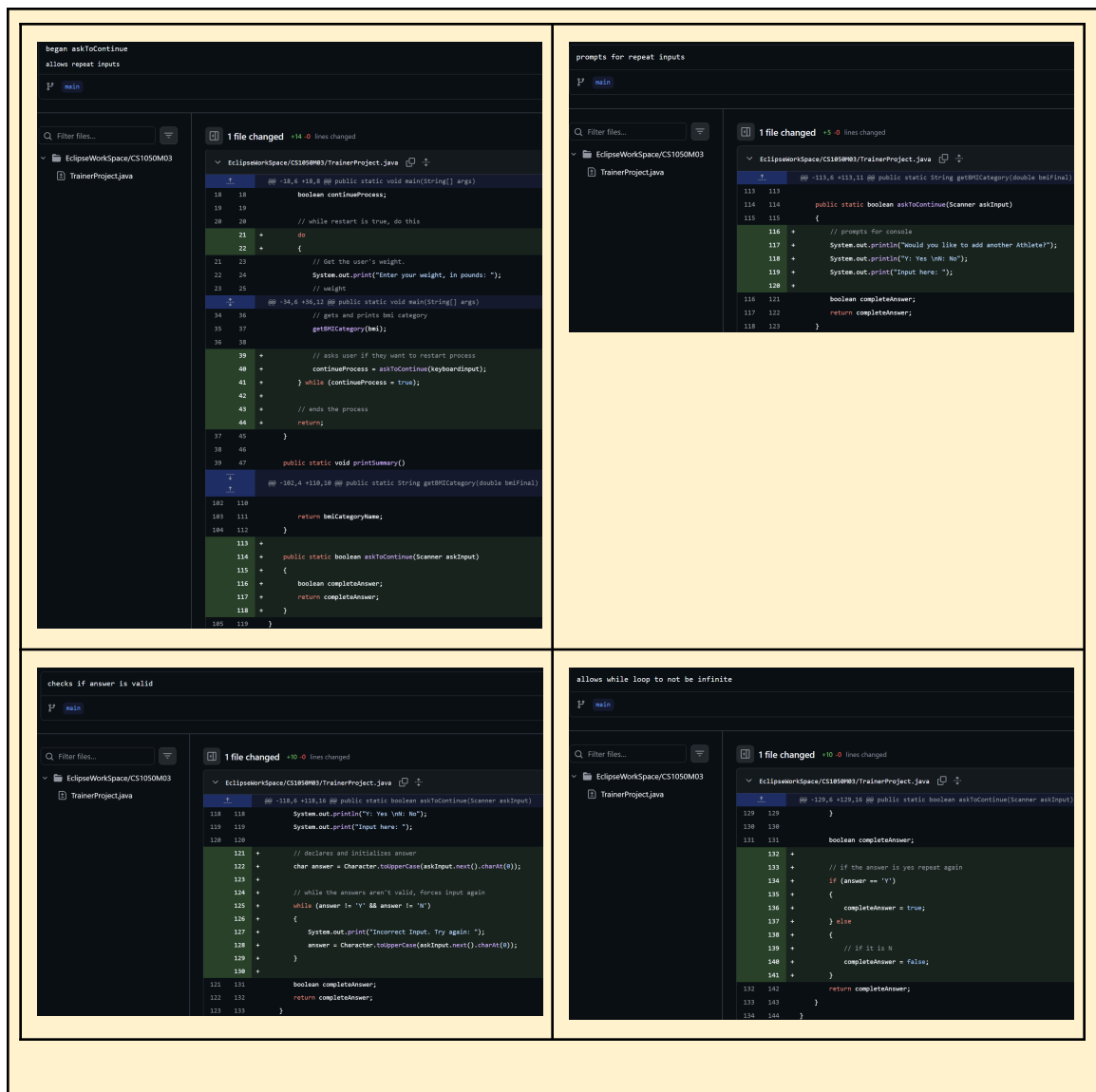
EclipseWorkspace/CS1050M03
  TrainerProject.java

1 file changed +10 0 lines changed
EclipseWorkspace/CS1050M03/TrainerProject.java
+++ @@ -38,6 +38,10 @@ public static void main(String[] args)
38
39 // calculate BMI/print
40 bmi = calculateBMI(weight, height);
41
42 +
43 + // gets and prints bmi category
44 + getBMICategory(bmi);
45 +
46 }
47
48 public static void printSummary()
49 {
50     @@ -70,4 +70,12 @@ public static double calculateBMI(double weightChosen, double heightChosen)
71     return bmiCalculated;
72 }
73
74 +
75 + public static String getBMICategory(double bmiFinal)
76 + {
77     // determines BMI Category
78     System.out.println("Your BMI category is " + bmiCategoryName);
79
80     return bmiCategoryName;
81 }
```

```
algorithm for bmiCategory
P main

EclipseWorkspace/CS1050M03
  TrainerProject.java

1 file changed +20 0 lines changed
EclipseWorkspace/CS1050M03/TrainerProject.java
+++ @@ -78,6 +78,28 @@
78
79 public static String getBMICategory(double bmiFinal)
80 {
81     // determines BMI Category
82     String bmiCategoryName;
83     if (bmiFinal >= 40)
84     {
85         // bmi >=40 means obese
86         bmiCategoryName = "Obese";
87     } else if (bmiFinal >= 35)
88     {
89         // bmi >=35 means overweight
90         bmiCategoryName = "Overweight";
91     } else if (bmiFinal >= 18.5)
92     {
93         // bmi >=18.5 means Normal
94         bmiCategoryName = "Normal";
95     } else
96     {
97         // bmi < 18.5 is underweight
98         bmiCategoryName = "Underweight: Needs Review";
99     } // and also
100     // prints bmiCategory
101     System.out.println("Your BMI category is " + bmiCategoryName);
102
103     return bmiCategoryName;
104 }
```



User Story 2: Method to Validate a Positive Double

Description: Prompt the user for value and repeats until the user enters a positive integer.

Create 4 test cases and include edge cases

Precondition	Algorithm Notes	Postcondition
Weight: 10 Height: -1	Weight >= 0 Height >= 0	Error: Height Negative
Weight: -10 Height: 23	Weight >= 0 Height >= 0	Error: Weight Negative
Weight: 0 Height: 10	Weight >= 0 Height >= 0	Error: Weight Zero
Weight: 100	Weight >= 0	Error: Height Zero

Height: 0	Height >= 0	
-----------	-------------	--

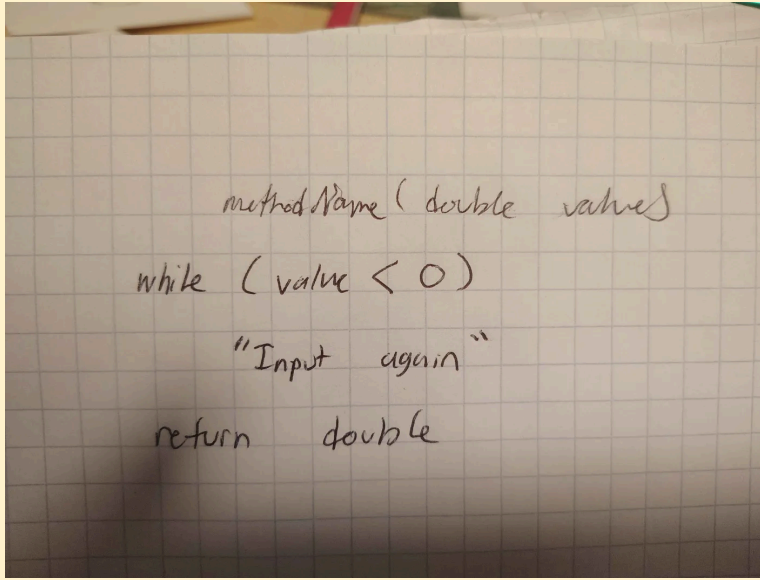
Method Name using verbs: GetPositiveDouble

Parameters Need Scanner object and String to display question prompt

Return: valid positive double

Method Signature: double GetPositiveDouble(Scanner methodInput, String questionPrompt)

Paste handwritten pseudocode algorithm below



Implement method code, call from main and test.

Optional: Add code that uses a scanner hasNext method to make sure a run time error doesn't occur if the user enters the incorrect data type.

User Story 3: Method to Calculate BMI

Description: Calculates BMI using formula: $703 \times \text{weight} / \text{height}^2$ where weight is in pounds and height is in inches

Create 4 test cases and include edge cases

Precondition	Algorithm Notes	Postcondition
Weight = 140 Height = 67	$703 \times 140 / 67^2$	21.9247
Weight = 140 Height = 72	$703 \times 140 / 72^2$	18.985
Weight = 180 Height = 69	$703 \times 180 / 69^2$	26.578

Weight = 49 Height = 20	$703 \times 49 / 20^2$	86.1175
----------------------------	------------------------	---------

Method Name (use verbs): calculateBMI

Parameters: double, double
Returns: double
Method Signature:

No need to write algorithms.

Implement method code, call from main and test.

User Story 4 Method: Get BMI Category Description: Based on the BMI value, determine the correct category value. Create test cases and include edge cases		
Precondition	Algorithm Notes	Postcondition
Bmi = 18.5	Bmi = 18.5 is	Normal
Bmi = 25	Bmi = 25	Overweight
Bmi = 17	Bmi < 18.5	Under
Bmi = 40	Bmi = 40	Obese
Bmi = 45	Bmi > 40	Obese
Bmi = 0	Error	Should be impossible given method getPositiveDouble

Method Name (use verbs): getBMICategory
Parameters (What does the method need): double bmi
Return (What does the method return): String
Method Signature (return type, method name, parameters): String
 getBMICategory (double currentBMI)

Paste handwritten pseudocode algorithm below

BMI ≥ 40	Obese
BMI ≥ 25	Overweight
≥ 18.5	Normal
< 18.5	Underweight / Needs review

Implement method code, call from main and test. Remember to commit and push.

User Story 5 Method: AskToContinue

Ask the user if they want to continue 'y' or 'n'.. Read the first character. When the user doesn't 'Y', 'y', 'N' or 'n' repeat the question. Return true for 'y' or 'Y' and false for 'n' or 'N'.

Create 4 test cases

Precondition	Algorithm Notes	Postcondition
Y	When (y) restart	Restarting
N	When (n) return	Ending Process
y	When (y) restart	Restarting
n	When (n) return	Ending Process
p	When (not y or n input2)	Error Invalid input

Method Signature: boolean AskToContinue()

Description: Ask the user if they want to continue. When the user doesn't enter a 'y', 'Y'

Paste handwritten pseudocode algorithm below

```

do
{
    every thing except summary

    "Would you like to repeat?" y/N

    char answer = Character.toLowerCase(keyboardInput.next().charAt(0));
    checkValidAnswer(answer);

    answer = while (answer == 'y');

    checkValidAnswer(answerMaybe)
    {
        while (answerMaybe != 'y' || answerMaybe != 'n')
        {
            "Invalid input. Try again /n Y: Yes /n N: No"
        }
        return answerMaybe;
    }
}

```

Implement method code, call from main and test. Remember to commit and push.

Main Method Planning

Think about what is the order of calling methods.
What is repeated and when?

The method is repeated when it is called in another method. First there is the method that the method being called is in, then the method being called occurs.

Does the loop at least run one time? Do you know how many times the loop will run?

Yes, the loop does run at least one time so that at least one athlete will be added. No, you do not know how many athletes are going to be added, therefore you will not know how many times it will need to run.

What type of loop would be best?

A do while loop because the number of athletes is not known and it must perform once.

Add this loop to the main method so the program will call the methods you created to work for one athlete.

Update your technical document to explain concepts and use examples from section 2.2.

3 Learnings and Reflection

For those that are advanced you can skip this part and complete [4 Advanced JUnit Exploration](#) instead.

1. Why was a while loop appropriate for allowing multiple athlete entries instead of a for loop?

It was appropriate to do while because we do not know the precise number of athletes a user wants to add.

2. Why is input validation important before performing calculations like BMI?

Input validation is important before performing calculations so that an impossible BMI is not calculated.

3. How did breaking the program into tasks make the implementation easier?

It made it easier to know which methods I should make first and what each method should do.

4. SRP and DRY

Explain SRP and include a good example that demonstrates this from 2.2.

Each method has its own purpose, the most concise one being the printSummary() method. In which, its only purpose is to print out a summary of the class to the console.

```
public static void printSummary()
{
    // Tell the user what the program will do.
    System.out.println(
        "This program will calculate your body mass index, or BMI using the
        imperial measurement system (pounds and inches).");
    System.out.println("There are 4 BMI Categories: " + "\n Obese: BMI = +40 " + "\n
    Overweight: BMI >= 25 "
        + "\n Normal: BMI >= 18.5 " + "\n Underweight: BMI < 18.5.");
    System.out.println("The BMI formula used is: weight * 703 / height^2.");
}
```

```
}
```

Explain DRY and include a good example that demonstrates this from 2.2.

Almost every method prevents the need to repeat code, but the most prominent is most likely `getPositiveDouble` because it allows you to use one method for both height and weight.

```
public static double getPositiveDouble(Scanner input, double bmiInput)
{
    bmiInput = input.nextDouble();
    // while the height is negative or 0, allows you to input again.
    while (bmiInput <= 0)
    {
        System.out.print("Invalid Input, cannot be negative or 0. Enter again: ");
        bmiInput = input.nextDouble();
    }
    return bmiInput;
}
```

5. Did any of your test cases reveal a logic error in your algorithm?
If so, explain.

What bug did you encounter while implementing this assignment?
How did you identify and fix it?

I kept running into this issue in the while loop inside of the continue process. It kept reading the `askToContinue` answer as 'Y' but the while loop kept. This was because I had an `or` statement comparing two variables. I fixed it by changing it to `and` `&&`. The reason why it was bugging is because when Y was inputted, it wasn't N and vice versa.

```
// while the answers aren't valid, forces input again
while (answer != 'Y' && answer != 'N')
{
    System.out.print("Incorrect Input. Try again: ");
    answer = Character.toUpperCase(askInput.next().charAt(0));
}
```